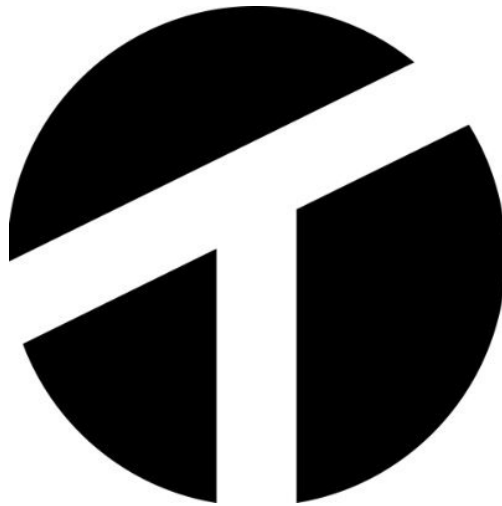


Othentic

Smart Contract Security Assessment

December 20, 2024



ABSTRACT

Dedaub was commissioned to perform a security audit for an upgrade to the Othentic protocol. In total, one medium issue and one low issue were identified, while several advisory notes, without direct implications for security, are also highlighted. The Othentic team also successfully remedied two issues highlighted in our original audit, namely [N1](#) and [A4](#).

BACKGROUND

Othentic is building a platform to enable the creation of Actively Validated Services (AVSs). In the original version of the protocol only one shared security provider was integrated, Eigenlayer, with this new version adding support for Symbiotic.

The architecture of the contracts has remained consistent with the prior version, with some changes to interfaces and flows to support these changes. A detailed description of the architecture can be found in the original audit report:

- <https://dedaub.com/audits/othentic/othentic-september-23-2024/>

SETTING & CAVEATS

The audit report mainly covers the contracts of the following [private repository](#) at commit number `f9da8492ba2f56e8815c419b2689f6e5b3ef85c7`.

2 auditors worked on the codebase for 3 days (each) on the following contracts:

`src/NetworkManagement`

- |— L1
 - | |— AvsGovernance.sol
 - | |— OthenticRegistry.sol
- |— L2
 - |— AttestationCenter.sol
 - |— InternalTaskHandler.sol

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

FIXES

Since the first audit performed by Dedaub the Othentic team has fixed a number of issues that were previously left unresolved. A summary of the fixes can be found below with a link to the issue in the original report.

ID	Description	Original Issue
F1	Share Syncing has been integrated into a system task	N1
The original version of the protocol delegated share syncing – the process of updating how much voting power an operator had based on their underlying balance – to a centralised role. This left the protocol open to an attack where the share syncer could maliciously update the balances to pass or reject any task they wanted. The new version of the protocol introduces the concept of system tasks. System tasks are still performed by operators like any other tasks, however these are specifically designed to assist in the operation of the AVS (in this case updating the share balances). This solution has been generalised, with AVS’s able to implement their own tasks via the <code>InternalTaskHandler</code>. <code>InternalTaskHandler:60-84</code> <pre>function _votingPowerUpdate(IAttestationCenter.TaskInfo memory _task) internal { InternalTaskHandlerStorageData storage _sd = _getStorage(); IOBLS _obls = _sd.obls; VotingPowerUpdate memory _update = abi.decode(_task.data, (VotingPowerUpdate)); uint _lastCommitBlockL1 = _sd.lastCommitBlockL1; uint _lastCommitBlockL2 = _sd.lastCommitBlockL2; if (_lastCommitBlockL1 >= _update.toBlockL1) { revert InvalidToBlockL1VsLastCommitBlockL1(_lastCommitBlockL1+1); } if (_lastCommitBlockL2 >= _update.toBlockL2) {</pre>		

```

        revert InvalidToBlockL2VsLastCommitBlockL2(_lastCommitBlockL2+1);
    }
    if (_update.toBlockL2 >= block.number) {
        revert InvalidToBlockL2VsCurrentHeight(_update.toBlockL2, block.number);
    }
    _sd.lastCommitBlockL1 = _update.toBlockL1;
    _sd.lastCommitBlockL2 = _update.toBlockL2;
    if (_update.toIncrease.length > 0) {
        _obls.increaseBatchOperatorVotingPower(_update.toIncrease);
    }
    if (_update.toDecrease.length > 0) {
        _obls.decreaseBatchOperatorVotingPower(_update.toDecrease);
    }
    emit VotingPowerUpdated(_update.toBlockL1, _update.toBlockL2,
        _task.proofOfTask);
}

```

F2

Removed special casing for default task

[A4](#)

The original version of the protocol included special handling for `taskDefinitionId` zero. The rationale behind this was to have a “default” task to speed up developer onboarding. We disliked this approach as special casing, especially for validation, generally leads to issues as the code evolves and the invariants are forgotten.

The Othentic team have now fixed these issues by implementing a default task which is created upon initialisation, and removed the special casing for `taskDefinitionId` zero.

PROTOCOL-LEVEL CONSIDERATIONS

[No protocol level considerations]

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes “dismissed” or “acknowledged” but no action taken, by the client, or “resolved”, per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

ID	Description	STATUS
M1	EigenLayer's Beacon Chain ETH strategy shares are not accounted	RESOLVED
In any AVS, an operator's voting power is calculated in terms of the amount of shares which have been delegated to them from a set of supported strategies. AvsGovernance:200 <pre>function votingPower(address _operator) external view returns (uint256) { ... return _getVotingPower(_sd, _operator, _votingPowerMultipliers); }</pre> AvsGovernance:545 <pre>function _getVotingPower(AvsGovernanceStorageData storage _sd, address _operator, IAvsGovernance.VotingPowerMultiplier[] memory _votingPowerMultipliers) private view returns (uint256) { ... uint256 _votingPower = _sd.othenticRegistry.getVotingPower(_operator, _votingPowerMultipliers, address(this)); ... return _votingPower; }</pre>		

Inside the `OthenticRegistry` contract, each strategy is checked by either EigenLayer's strategy manager or Symbiotic's vault factory to determine whether it's a valid strategy. If a strategy is deemed unsupported, the operator is considered to have 0 voting power based on that strategy.

`OthenticRegistry:213`

```
function _getVotingPower(address _operator, IAvsGovernance.VotingPowerMultiplier
calldata _votingPowerMultiplier, address _avsGovernance) private view returns
(uint256) {
    if(!_isValidStakingContract(_votingPowerMultiplier.stakingContract)) {
        return 0;
    }
    ...
}
```

`OthenticRegistry:244`

```
function _isValidStakingContract(address _stakingContract) private view returns (bool) {
    return strategyManager.strategyIsWhitelistedForDeposit(IStrategy(_stakingContract)) ||
    vaultFactory.isEntity(_stakingContract);
}
```

One of the EigenLayer strategies that is supported by default in newly setup AVSs is the Beacon Chain ETH strategy

`OthenticRegistry:244`

```
address private constant BEACON_CHAIN_ETH_STRATEGY =
0xbeaC0eeEeeeeEeeEeEEEEeeEeeEeeEeeEEBEaC0;
...
function getDefaultStrategies(uint _chainid) external pure returns
(IAvsGovernance.StakingContractInfo[] memory) {
    if (_chainid == 1) {
        IAvsGovernance.StakingContractInfo[] memory _strategies = new
        IAvsGovernance.StakingContractInfo[](14);
        ...
        _strategies[12] =
        IAvsGovernance.StakingContractInfo(BEACON_CHAIN_ETH_STRATEGY,
```



```
IAvsGovernance.SharedSecurityProvider.EigenLayer); // Beacon Chain ETH strategy  
...
```

The problem lies in the fact the strategy manager (ETH:0x858646372CC42E1A627fcE94aa7A7033e7CF075A) does not consider the Beacon Chain ETH strategy as whitelisted for deposit, meaning that `strategyIsWhitelistedForDeposit` will return false when attempting to calculate an operator's voting power based on that strategy. The core EigenLayer contracts handle this address as a sentinel value, and the entity that records deposits and withdrawals on it is the `EigenPodManager`, which is what the `DelegationManager` contract consults in order to determine an operator's shares on the Beacon Chain ETH strategy.

As a consequence, `0thenticRegistry::getVotingPower` will return 0 for this strategy and so the protocol's accounting will not consider any voting power coming out of the Beacon Chain ETH.

LOW SEVERITY:

ID	Description	STATUS
L1	Race conditions upon an AVS's initialization	ACKNOWLEDGED
Comments: The deployed AVSs are, by default, permissionless, meaning there are no restrictions on: • The operators that can opt-in to an AVS • The minimum voting power of the participating operators While the considerations of this issue are acknowledged, AVSs that require such restrictions as part of their business logic are expected to enforce them immediately after initialization. In order to guarantee that the limits are set atomically, the owner of an AVS should use appropriate transaction bundling or sequencing tools (e.g., Flashbots). When an <code>AvsGovernance</code> contract is initialized, there are some state parameters that are not initialized directly in <code>AvsGovernance::_initialize</code>: <code>AvsGovernance::_initialize:95</code> <pre> ... AvsGovernanceStorageData storage _sd = _getStorage(); _sd.othenticRegistry = _othenticRegistry; _sd.messageHandler = IMessageHandler(_messageHandler); _grantRole(RolesLibrary.MESSAGE_HANDLER, _messageHandler); _sd.avsTreasury = IAvsTreasury(_initializationParams.avsTreasury); _sd.allowlistSigner = _initializationParams.allowlistSigner; _sd.rewardsReceiverModificationDelay = 7 days; _sd.avsDirectoryContract = IAVSDirectory(_initializationParams.avsDirectoryContract); _sd.numOfOperatorsLimit = 100; </pre>		

```
_sd.blsAuthSingleton = _initializationParams.blsAuthSingleton;  
_setAvsName(_sd, _avsName);  
...
```

For the `minStakeAmountPerStakingContract` mapping as well as the `minVotingPower` and `isAllowlisted` state variables, an appropriate setter function must be used.

Because the corresponding setter functions can only be called by the AVS Governance Multisig, and those parameters cannot be set atomically during initialization, there may arise some race conditions after the contract's initialization:

- With `minVotingPower` and `minStakeAmountPerStakingContract` being unset, an operator may register without having any shares. This could lead to scenarios where someone bundles multiple transactions (using different contracts or `tx.origins`) right after the contract's initialization to register as an operator. This could lead to a temporary DOS where the `numOfOperatorsLimit` is hit and legitimate operators are temporarily unable to register (until the limit is increased)

`AvsGovernance::_registerAsOperator:448`

```
...  
bool _isActive = (_votingPower >= _sd.minVotingPower) &&  
_sd.othenticRegistry.isValidStakeAmount(_operator, _minStakePerStakingContract,  
address(this));  
if (!_isActive) revert NotEnoughVotingPower();  
...
```

- Both with and without the above-mentioned limits being set, someone can always front-run calls to `AvsGovernance::setIsAllowlisted` and register before the whitelisting functionality is turned on

AvsGovernance :114

```
function registerAsOperator(OperatorRegistrationParams calldata
_operatorRegistrationParams) ... {
    AvsGovernanceStorageData storage _sd = _getStorage();
    if (_sd.isAllowlisted) {
        if (_operatorRegistrationParams.authToken.length == 0) {
            revert MissingAuthToken(_operatorRegistrationParams.authToken);
        }
        ...
    }
    ...
}
```

It should be noted that transactions using the setters are susceptible to front-running unless bundled with a service like Flashbots, even if sent shortly after the contract's initialization.

One way to mitigate these considerations altogether is to allow the corresponding storage fields to be set atomically during the contract's initialization.

CENTRALIZATION ISSUES:

It is often desirable for DeFi protocols to assume no trust in a central authority, including the protocol's owner. Even if the owner is reputable, users are more likely to engage with a protocol that guarantees no catastrophic failure even in the case the owner gets hacked/compromised.

We have no additional centralization concerns since our previous audit. We highlight that in the version that was audited during this engagement our previous centralization concern has been fully addressed:

- <https://dedaub.com/audits/othentic/othentic-september-23-2024/#n1>

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Inconsistent voting power calculation	RESOLVED
When registering as an operator in the <code>AVSGovernance</code> via <code>registerAsOperator</code>, operators are first expected to call <code>registerOperatorToEigenLayer</code> and/or <code>registerOperatorToSymbiotic</code>. If an operator fails to call <code>registerOperatorToEigenLayer</code> prior to <code>registerAsOperator</code> its votes will still be calculated correctly, as it calculates the share amount from the <code>DelegationManager</code> (via the <code>0thenticRegistry</code>) which does a direct storage lookup. However if an operator fails to call <code>registerOperatorToSymbiotic</code> prior to <code>registerAsOperator</code> it will have zero voting power, as <code>BaseDelegator.stakeAt</code> consults the <code>OptInService</code> to determine if the operator has registered with the AVS and will return zero in the case it has not been registered.		
A2	Enums can cause issues for non-upgradeable contracts	ACKNOWLEDGED
Using an enum to represent <code>SharedSecurityProvider</code> in <code>AvgGovernance</code> can lead to compatibility issues with non-upgradeable contracts if the enum is ever expanded. This is because the compiler automatically inserts range checks for enum values, regardless of how the enum value is used. As a result, adding new enum values later can cause existing non-upgradeable contracts to fail when they encounter values outside the original range.		

Comments:

This finding does not directly impact the Othentic protocol as all the contracts interacting with this enum are upgradeable. We have chosen to leave this finding in the report as advice to any developers planning to integrate with Othentic, and we would also like to note that this is not specific to Othentic but to Solidity enums generally.

A3	The empty string is a valid AVS name on-chain	RESOLVED
----	---	----------

AvsGovernance:106

```
function _initialize(InitializationParams calldata _initializationParams) internal
onlyInitializing {
    ...
    _othenticRegistry.registerAvs(_avsName);
    ...
}
```

OthenticRegistry:79

```
function registerAvs(string memory _avsName) external {
    emit AvsOptIn(msg.sender, _avsName);
}
```

Although there are no security consequences on-chain nor off-chain, developers might consider disallowing the empty string from being a valid AVS name.

A4	Functions can be made external	RESOLVED
----	--------------------------------	----------

The following functions are not used internally anywhere inside the **AvsGovernance** contract and thus can be marked external

AvsGovernance:310

```
function setMinStakesForStakingContract(address _stakingContract, uint256
_minShares) public ... {
```

AvsGovernance:356

```
function setStakingContractMultiplierBatch(VotingPowerMultiplier[] calldata
_votingPowerMultipliers) public ... {
```

A5

Unnecessary initialization with 0

RESOLVED

The following initialization is not necessary

AttestationCenter:77

```
function _initialize(InitializationParams calldata _initializationParams) internal
onlyInitializing {
...
    _sd.numOfTotalOperators = 0;
...
}
```

A6

Unconventional storage gap size

RESOLVED

The storage gap in the `OthenticRegistry` contract is 45 slots

`OthenticRegistry:248`

```
// slither-disable-next-line unused-state,naming-convention
uint256[45] private __gap;
```

However, given the contract's storage layout and the convention of allocating a total of 50 slots in gap storage, the developers could consider making the current gap be 41 slots.

`OthenticRegistry:36`

```
contract OthenticRegistry is Initializable, OwnableUpgradeable, IOthenticRegistry
{
    ...

    address private constant BEACON_CHAIN_ETH_STRATEGY =
0xbeaC0eeEeeeeEEeEeEEEEeEeEeeEeeEEBEaC0;

    //Eigenlayer contracts
    ISlasher public slasher;
    IDelegationManager public delegationManager;
    IStrategyManager public strategyManager;

    //Othentic contracts
    AvsGovernances private avsGovernances; // obsolete

    //Symbiotic contracts
    IVaultFactory public vaultFactory;
    IOptInService public optInService;
    INetworkRegistry public networkRegistry;
    INetworkMiddlewareService public networkMiddlewareService;

    //Othentic contracts
    IL1AvsFactory public l1AvsFactory;
```

A7	Compiler bugs	INFO
The code is compiled with Solidity version 0.8.25 which has no known bugs .		

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.